H.11 Approved procedure update

Procedure update: 11 October 2026

This records the user's approved change to SFT corpus generation and the executed storage and RL preparation steps. It amends the earlier native Transformers SFT generation procedure. Training recipes, GRPO rollout generation, evaluation settings, acceptance rules, and statistical gates retain their existing definitions.

SFT corpus generation

Use separate H100 80-GB Pods for the untouched pinned 7B and 14B models. Each runs `scripts/run_vllm_generation_worker.py` from frozen generation commit `dc520cb73bf43b59a9566404dd4a7aa8358f1cc3`. Both consume the same ordered, audited initial 10,100 LMSYS prompt IDs. The configuration hash is `3757493e93d7f3ac31d9215bd493ae4f13cae0bed7c97e773f4ca4ef5be2ed54`; run ID is `vllm_fresh_dc520cb`.

The approved backend is vLLM 0.11.0 with Transformers 4.57.3 and Torch 2.8.0+cu128. Use BF16 weights without quantization, 128 requests per chunk, maximum model length 4096, maximum concurrent sequences 128, scheduler token budget 8192, GPU memory utilization 0.9, and prefix caching disabled. The scheduler token budget is concurrent processing capacity, not a shared output allowance: **every request retains its own 3072-new-token cap**. Temperature remains 0.6 and top_p remains 0.95. Preserve native chat-template prompt token IDs, EOS IDs, pinned model/tokenizer revisions, and independently derived per-example seeds. Prompt eligibility remains at most 200 input tokens.

The vLLM environment is separate from the native Torch 2.9 training environment. Its actual installed package inventory and hash are saved in each immutable run manifest; the native dependency lock is not claimed to describe this engine. Identical weights and sampling parameters do not imply bitwise-identical sampled outputs across engines.

Generate the corpus afresh from prompt one. Old native responses and benchmark responses remain archived and are excluded from the fresh training corpus. Both training models must use only matching fresh vLLM production responses. This is a generation backend change, not a change to the SFT or RL optimizer.

Drive is the permanent store

The central project folder is Inducing Persistent Textual Chain-of-Thought Illegibility. RunPod container disks are disposable scratch; no Pod volume or network volume is relied upon for preservation.

The worker saves raw responses locally, registers each response with its source/configuration identity, uploads through the repository ArtifactService, and verifies the Drive copy before starting another batch. Immutable run metadata is uploaded before GPU work. Reported progress counts only batches that passed this Drive barrier.

Artifact	Folder within the central project
Fresh 7B responses and metadata	`collect_clean/shared/7b/vllm_fresh_dc520cb` (open)
Fresh 14B responses and metadata	`collect_clean/shared/14b/vllm_fresh_dc520cb` (open)
Shared RL question manifest	`prepare_rl/shared/shared/rl_questions_800` (open)

Intentional pauses drain a whole batch. Restart only after the prior worker has exited, using the original source, package inventory, arguments, configuration, and output roots. The worker checks existing raw identities and skips compatible saved

responses; pending uploads must be verified before new generation. The observed 7B interruption was two Drive uploadtimeouts, not a CUDA memory failure; resuming repaired those uploads without regenerating saved responses. Stopping aworker does not stop Pod billing; the account operator controls the Pod console.

An authenticated Jupyter connection operates each Pod; RunPod account API keys are unavailable under the PI account's permissions. Google Drive uses the existing restricted drive.file OAuth authorization. Credentials remain in protected files outside Git and experimental artifacts. Do not put tokens in documentation, manifests, or connection logs.

Raw generation is not final dataset acceptance

10,100 initial raw responses per model are not 10,100 accepted pairs. Once the two caches are complete, join by the original prompt ID and apply the existing structural completion, reasoning/final parsing, eligible-word, protection, and clean/all-transformed length checks. Do not add semantic correctness or quality filtering.

Refills use new audited IDs in fixed order, with the batch size exactly equal to the accepted-pair deficit. Rejected ordinary completions are retained rather than repeatedly sampled. Generate refill responses through separate model-specific vLLM workers with recorded provenance; the existing single-model engine cannot load both models in one process. Refill orchestration is still pending. Freeze the shared 10,000 training and 100 validation IDs only after enough pairs pass all checks. Then construct and verify all ROT13 and KaziText condition datasets and their Drive copies. Never mix old native or benchmark responses into these caches.

Evaluation and RL data

The frozen 1,233-item capability panel and nested 200-item legibility subset were prepared and backed up before the backend change. Their IDs are reused unchanged; actual model evaluations are pending. The SFT generation engine change does not alter native evaluation decoding or judging.

The repository's `prepare_rl_data` function has now selected 800 unique GSM8K training questions from revision `740312add88f781978c0658806c59bc2815b9866`. It checked valid numeric references, duplicate questions, and overlap against the complete official evaluation frames, then fixed seed-42 ordering into 100 consecutive groups of eight. Manifest content hash: `bc4c6b1b50906a9ee1c3ee47c9f4a6b72a7dd6dacc852ac58a1b090688b4a0a8`. Preparation source commit: `a03a70dbd520a34cf0dda67e73d0de2337a87140`. The manifest was uploaded and verified in the shared RL folder linked above.

Both RL trajectories use this same ordered manifest. Preparation does not create imitation reasoning targets or policy responses. The current policy generates four rollouts per question during GRPO; eight questions and 32 rollouts form one logical update. The existing RL temperature, token cap, reward rules, optimizer, and checkpoint cadence remain unchanged. RL training has not started.

Validation and monitoring

The vLLM benchmark checked actual generation, formatting, token limits, and GPU memory on the existing hardware. Matched 14B batches of 32 took approximately 40.53 seconds with vLLM versus 342.78 seconds with native generation; the 128-request vLLM sample took 52.64 seconds without OOM. These are generation-only samples, not quality equivalence or end-to-end throughput guarantees. Every production batch separately passes Drive integrity verification.

At the user's request, the full test suite and full launch preflights were not repeated for the engine migration. Final acceptance, cross-model pairing, transformed datasets, and training remain unverified. Prior prospective statistical validation reported 34/36 passing scenarios with two failures; this update does not waive that training/confirmatory launch blocker.

`scripts/monitor_vllm_generation.py` checks both Pods every 30 seconds, records safe status snapshots locally, and reports progress, terminal events, connection errors, and ten-minute stalls. It does not restart workers or send

autonomous chat notifications. Its ETA covers initial raw generation and Drive replication only, excluding refills, transformations, training, and evaluation.

The active generation source remains frozen even when documentation or the monitor is pushed to newer GitHub commits. Repeated test counts and transient generation totals are not presented as completed scientific results.

Updated workflow sources and PDFs

The active clean collection diagram, candidate batch diagram, and RL preparation diagram describe the updated procedure. Their module documents mirror the Mermaid sources.

procedure_update.pdf is the standalone H.11 amendment generated from this document. The same amendment is appended to experiment.pdf, preserving the preceding 81 pages and existing page citations. The 67-page historical source in `spec/archive/experiment_original.pdf` remains unchanged. Rebuild both active PDFs with `python scripts/build_procedure_update_pdf.py` using ReportLab and pypdf.